



Alma Mater Studiorum-Università di Bologna Scuola di Ingegneria

Lambda expression: casi applicativi

Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

Un cruciale caso applicativo Back to Comparator



UN ANTICO ESEMPIO: ORDINARE UN ARRAY DI PERSONE

- Consideriamo una variante della cara vecchia classe **Persona**:

```
class Persona {  
    private String nome, cognome;  
    private int età;  
  
    public Persona(String nome, String cognome, int età) {  
        this.nome=nome; this.cognome=cognome; this.età=età;  
    }  
  
    public String getCognome(){ return cognome; }  
    public String getNome(){ return nome; }  
    public int getEtà(){ return età; }  
}
```

Java

- Consideriamo inoltre un classico array di **Persona**:

```
Persona[] persone = new Persona[]{  
    new Persona("John", "Doe", 25),  
    new Persona("Jane", "Doe", 45),  
    new Persona("Anne", "Bee", 31),  
    new Persona("Jane", "Doe", 22)  
};
```

Java



UN ANTICO ESEMPIO: ORDINARE UN ARRAY DI PERSONE

- Si possono definire vari comparatori che incapsulino diversi criteri:

```
class CognomeComparator implements Comparator<Persona> {  
    public int compare(Persona p1, Persona p2){  
        return p1.getCognome().compareTo(p2.getCognome());  
    }  
}  
  
class NomeComparator implements Comparator<Persona> {  
    public int compare(Persona p1, Persona p2){  
        return p1.getNome().compareTo(p2.getNome());  
    }  
}  
  
class EtaComparator implements Comparator<Persona> {  
    public int compare(Persona p1, Persona p2){  
        return Integer.compare(p1.getEtà(), p2.getEtà());  
    }  
}
```

Java



UN ANTICO ESEMPIO: ORDINARE UN ARRAY DI PERSONE

- Un classico mini-main di esempio che ordini l'array in vari modi:

```
System.out.println(String.join(" , " , persone) ;  
  
Arrays.sort(persone, new CognomeComparator() ) ;  
System.out.println(String.join(" , " , persone) ;  
  
Arrays.sort(persone, new NomeComparator() ) ;  
System.out.println(String.join(" , " , persone) ;  
  
Arrays.sort(persone, new EtaComparator() ) ;  
System.out.println(String.join(" , " , persone) ;
```

Java

- Avevamo anche considerato il caso del comparatore definito tramite *classe anonima*, inline:

```
Arrays.sort(persone, new Comparator<Persona>() {  
    public int compare(Persona p1, Persona p2) {  
        return p1.getCognome().compareTo(p2.getCognome()) ;  
    }  
} ;
```

Java



IL RUOLO DEL Comparator

- Riflettendo.. **qual è esattamente il ruolo del Comparator?**
- A ben vedere, è solo quello permetterci di passare a **sort** la **funzione di confronto (compare)**

```
Arrays.sort(persone,  
    new Comparator<Persona>() {  
        public int compare(Persona x, Persona y){...}  
    }  
);
```

Java

- **Comparator** è una sorta di «wrapper» per la **compare**
 - non essendo le funzioni delle vere first-class entity, in Java **compare** *non può essere manipolata come oggetto stand alone*
 - il comparator le fa quindi da «adulto accompagnatore», perché lei non può «andare in giro da sola»



PASSARE COMPORTAMENTO: ANCORA IL COMPARATOR

- L'interfaccia **Comparator** ha in effetti come suo *unico* scopo quello di dichiarare il metodo **compare**
 - non a caso, è il suo *unico* metodo
- Implementando **Comparator**, noi implementiamo di fatto *una specifica funzione compare*
 - è quella che vorremmo "passare" a **sort** ...
 - ...ma che finora non abbiamo potuto passarle direttamente
- Non sarebbe più bello se potessimo scrivere qualcosa come:

```
Arrays.sort(persone, compare(...)) ;
```

senza tutto quell'accrocchio intorno...? 😊



PASSARE COMPORTAMENTO: OLTRE IL COMPARATOR

- Quello che abbiamo fatto è passare a `sort` *un Comparator che incapsula la «giusta» compare*
- MA il comparatore è solo un veicolo: ciò che in realtà serve alla `sort` è *solo la compare!*
- Possiamo farlo introducendo opportune *lambda expression* che equivalgano ad altrettanti «comparatori anonimi»



UN ANTICO ESEMPIO: ORDINARE UN ARRAY DI PERSONE

- L'esempio di poco fa, con comparatori classici:

```
System.out.println(String.join(" , " , persone) ;  
  
Arrays.sort(persone, new CognomeComparator() ) ;  
System.out.println(String.join(" , " , persone) ;  
  
Arrays.sort(persone, new NomeComparator() ) ;  
System.out.println(String.join(" , " , persone) ;  
  
Arrays.sort(persone, new EtaComparator() ) ;  
System.out.println(String.join(" , " , persone) ;
```

Java

- L'esempio con comparatore anonimo:

```
Arrays.sort(persone, new Comparator<Persona>() {  
    public int compare(Persona p1, Persona p2) {  
        return p1.getCognome().compareTo(p2.getCognome()) ;  
    }  
} ;
```

Java



ESEMPIO: ORDINARE UN ARRAY DI PERSONE

- La lambda expression può fare la stessa cosa con eleganza:
ogni lambda equivale a un comparatore anonimo

```
// versione con lambda expression
System.out.println(Arrays.asList(persone));
Arrays.sort(persone,
    (p1,p2) -> p1.getCognome().compareTo(p2.getCognome()) );
System.out.println(Arrays.asList(persone));

Arrays.sort(persone,
    (p1,p2) -> Integer.compare(p1.getEtà(), p2.getEtà()) );
System.out.println(Arrays.asList(persone));
```

Java

equivale a

```
Comparator<Persona> cmp = new Comparator<Persona> {
    public int compare(Persona p1, Persona p2){
        return p1.getCognome().compareTo(p2.getCognome());
    }
}
```



ESEMPIO: ORDINARE UN ARRAY DI PERSONE

- La lambda expression può fare la stessa cosa con eleganza:
ogni lambda equivale a un comparatore anonimo

```
// versione con lambda expression
System.out.println(Arrays.asList(persone));
Arrays.sort(persone,
    (p1,p2) -> p1.getCognome().compareTo(p2.getCognome()) );
System.out.println(Arrays.asList(persone));

Arrays.sort(persone,
    (p1,p2) -> Integer.compare(p1.getEtà(), p2.getEtà()) );
System.out.println(Arrays.asList(persone));
```

Java

equivale a

```
Comparator<Persona> cmp = new Comparator<Persona> {
    public int compare(Persona p1, Persona p2){
        return Integer.compare(p1.getEtà(), p2.getEtà());
    }
}
```

UN PASSO IN PIÙ

- Non è finita: osservando bene, si nota che la **struttura** delle varie lambda-comparatori è **sostanzialmente identica**
 - tutti i comparatori confrontano due oggetti, restituendo +1, -1 o 0
 - l'unica cosa che cambia è **il valore da confrontare**
 - nel primo caso, *il cognome* → metodo **getCognome**
 - nel secondo caso, *l'età* → metodo **getEtà**

```
System.out.println(Arrays.asList(persone));  
Arrays.sort(persone,  
    (p1,p2) -> p1.getCognome().compareTo(p2.getCognome()));  
System.out.println(Arrays.asList(persone));  
  
Arrays.sort(persone,  
    (p1,p2) -> Integer.compare(p1.getEtà(), p2.getEtà()));  
System.out.println(Arrays.asList(persone));
```

Java



UN COMPARATORE «GENERALE»

- In effetti, ogni comparatore ha una struttura fissa: *cambia solo la **proprietà** su cui si effettua il confronto*

$(x, y) \rightarrow x.\text{getProp}().\text{compareTo}(y.\text{getProp}())$

- Ma allora.. se fattorizziamo in una piccola lambda (***estrattore***) la **proprietà di interesse** per il confronto:

$x \rightarrow x.\text{getProp}()$

- Il comparatore può essere ***sintetizzato da una factory*** che riceva appunto *l'estrattore* della proprietà di interesse!

`Comparator.comparing(estrattore)`



LA FABBRICA DEI COMPARATORI

- Il metodo statico **comparing** è la factory dei comparatori
 - si chiama così perché la frase "viene bene in inglese"
 - riceve l'estrattore (mini-lambda) della *proprietà da confrontare*

PRIMA:

```
Arrays.sort(persone,  
    (p1,p2) -> p1.getCognome().compareTo( p2.getCognome( ) ) );
```

Java

DOPO:

```
Arrays.sort(persone, Comparator.comparing( p -> p.getCognome( ) ) );
```

- in inglese: sort «by means of a comparator *comparing* this property»
 - in questo caso: ordina «*confrontando per cognome*»
- Ciliegina sulla torta: **l'estrattore come *method reference*** 😊

```
Arrays.sort(persone, Comparator.comparing( Persona::getCognome ) );
```



COMPARATORS, AT LAST!

- Ora la transizione è completa
 - anziché definire una classe-comparatore ad hoc, da istanziare appositamente per quella **sort**:

```
class CognomeComparator implements Comparator<Persona> {  
    public int compare(Persona p1, Persona p2){  
        return p1.getCognome().compareTo(p2.getCognome());  
    }  
}  
Arrays.sort(persone, new CognomeComparator());
```

Java

- anziché l'intrico innestato delle classi anonime
- *un approccio pulito, auto-documentante e leggibile*, in cui si riconosce subito cosa si fa

```
Arrays.sort(persone,  
    Comparator.comparing( Persona::getCognome ) ;
```

Java



COMPARATORI PER TIPI PRIMITIVI

- Anche in questo caso, come sempre in Java, i tipi primitivi, purtroppo, richiedono un trattamento ad hoc
 - tre factory method specifici per i tre più usati: `int`, `long` e `double`
 - `comparingInt`, `comparingLong`, `comparingDouble`

PRIMA:

Java

```
Arrays.sort(persone,  
    (p1,p2) -> Integer.compare( p1.getEtà() , p2.getEtà() ));
```

DOPO:

```
Arrays.sort(persone,  
    Comparator.comparingInt( p -> p.getEtà() ));
```

COMPARATORI MULTIPLI

- Si possono perfino applicare *criteri di ordinamento multipli*
 - prima un criterio principale (ad esempio, ordinamento per cognome)
 - poi un criterio secondario (ad esempio, ordinamento per nome)
 - poi un criterio ulteriore (ad esempio, ordinamento per età)
- Basta *combinare più comparatori* con *thenComparing*

```
Arrays.sort(persone,  
    Comparator.comparing( Persona::getCognome )  
                .thenComparing( Persona::getNome )  
                .thenComparingInt( Persona::getEtà ) ;
```

Java

- *Fluent interface* ottenuta tramite *cascading*:
confronta *per cognome* e in *subordine per nome* e poi *per età*
- Funziona perché **comparing** restituisce il **Comparator** creato,
usato poi come target dalla successiva **thenComparing**



...BECAUSE READABILITY MATTERS

- Giusto per apprezzare la differenza...
 - con factory, lambda come extractor e method reference:

```
Arrays.sort(persone,  
    Comparator.comparing( Persona::getCognome )  
                .thenComparing( Persona::getNome )  
                .thenComparingInt( Persona::getEtà ) );
```

Java

- la stessa cosa con l'approccio classico:

```
class MyComparator implements Comparator<Persona> {  
    public int compare(Persona p1, Persona p2) {  
        boolean res = p1.getCognome().compareTo(p2.getCognome());  
        if (res!=0) return res;  
        res = p1.getNome().compareTo(p2.getNome());  
        if (res!=0) return res;  
        else return Integer.compare( p1.getEtà(), p2.getEtà() )  
    }  
}
```

Java



...BECAUSE READABILITY MATTERS

- Giusto per apprezzare la differenza...
 - con factory, lambda come extractor e method reference:

```
Arrays.sort(persone,  
    Comparator.comparing( Persona::getCognome )  
        .thenComparing( Persona::getNome )  
        .thenComparingInt( Persona::getEtà ) );
```

- Espressività
- Estendibilità
- Manutenibilità

- la stessa cosa con l'approccio classico:

```
class MyComparator implements Comparator<Persona>  
    public int compare(Persona p1, Persona p2) {  
        boolean res = p1.getCognome().compareTo(p2.getCognome());  
        if (res!=0) return res;  
        res = p1.getNome().compareTo(p2.getNome());  
        if (res!=0) return res;  
        else return Integer.compare( p1.getEtà(), p2.getEtà() )  
    }  
}
```

- Cripticità
- Manutenibilità faticosa



COMPARATORI MULTIPLI: LIMITI

- ATTEZIONE: talora la type inference *può non farcela* a dedurre con certezza il tipo della lambda se *il contesto non dà sufficienti informazioni*
- Ad esempio, mentre questa compila regolarmente:

```
Collections.sort(playlist, comparing(p1 -> p1.getTitle()));
```

il comparatore multiplo seguente *non compila*:

```
Collections.sort(playlist,  
    comparing(p1 -> p1.getTitle())  
    .thenComparing(p1 -> p1.getDuration())  
    .thenComparing(p1 -> p1.getArtist())    );
```

Java

- MOTIVO: nel primo caso, il tipo del risultato di `comparing` è ricavato dalla signature di `Collections.sort`: poiché `playlist` è `List<Song>`, `comparing` deve fornire un `Comparator<Song>`, quindi `p1` dev'essere una `Song`.

COMPARATORI MULTIPLI: LIMITI

- ATTENZIONE: talora la type inference *può non farcela* a dedurre con certezza il tipo della lambda se *il contesto non dà sufficienti informazioni*
- Ad esempio, mentre questa compila regolarmente:

```
Collections.sort(playlist, comparing(p1 -> p1.getTitle()));
```

il comparatore multiplo seguente *non compila*:

```
Collections.sort(playlist,  
    comparing(p1 -> p1.getTitle())  
    .thenComparing(p1 -> p1.getDuration())  
    .thenComparing(p1 -> p1.getArtist())    );
```

Java

- MOTIVO: nel secondo caso, `Comparator<Song>` è il tipo del risultato *dell'intera catena*, ma perché funzioni occorre *riuscire a inferire tutti i singoli pezzi*.
- Ora, il metodo `thenComparing` è invocato sul risultato di `comparing`, ma *poiché questa è generica*, non se ne conosce il tipo: quindi, non si può essere certi neppure del fatto che *abbia un metodo thenComparing*!

COMPARATORI MULTIPLI: LIMITI

- Per uscirne bisogna *aggiungere in qualche modo informazione di tipo*

```
Collections.sort(playlist, // non compila!  
    comparing(p1 -> p1.getTitle())  
        .thenComparing(p1 -> p1.getDuration())  
        .thenComparing(p1 -> p1.getArtist())    );
```

Java

- Ci sono fondamentalmente tre possibilità:
 - 😊😊 usare una method reference specifica, come **Song::getTitle**
 - 😊 usare una lambda con argomenti tipizzati, come **(Song p1) -> ...**
 - 😞 fornire la specifica di tipo in sede di chiamata di **comparing**, scrivendo **Comparator.<Song,String>comparing(...)**



COMPARATORI MULTIPLI: LIMITI

- ☺ ☺ usare una method reference specifica in tipo:

```
Collections.sort(playlist,  
    comparing(Song::getTitle)  
    .thenComparing(p1 -> p1.getDuration())  
    .thenComparing(p1 -> p1.getArtist())    );
```

Java

- ☺ Usare una lambda con argomenti tipizzati

```
Collections.sort(playlist,  
    comparing((Song p1) -> p1.getTitle())  
    .thenComparing(p1 -> p1.getDuration())  
    .thenComparing(p1 -> p1.getArtist())    );
```

Java

- ☹ fornire la specifica di tipo in sede di chiamata di **comparing**:

```
Collections.sort(playlist,  
    Comparator.<Song,String>comparing(p1 -> p1.getTitle())  
    .thenComparing(p1 -> p1.getDuration())  
    .thenComparing(p1 -> p1.getArtist())    );
```

Java



E NEGLI ALTRI LINGUAGGI...?

- In C#

- la fabbrica dei comparatori è `Comparer<T>.Create`
- l'ordinamento *su nuovo array* via lambda lo fa il metodo `OrderBy`

```
Array.Sort(persone, new MyComp()); // approccio classico
```

C#

```
Array.Sort(persone, Comparer<Persona>.Create(  
    (a,b) => a.GetNome().CompareTo(b.GetNome()) ));
```

```
IOrderedEnumerable<Counter> orderedArray =  
    persone.OrderBy( p => p.GetNome() );
```



E NEGLI ALTRI LINGUAGGI...?

- In Scala

- non c'è una «fabbrica di comparatori», si usano appositi metodi
- l'ordinamento «in place» via lambda lo fa il metodo `sortWith`
- l'ordinamento *su nuovo array* via lambda lo fa il metodo `sortBy`

```
Sorting.quickSort(persone) (MyComp); // classico
```

Scala

```
Sorting.quickSort(persone) (new Ordering[Persona] {  
    def compare(a: Persona, b: Persona) : Int =  
        a.getNome() compare b.getNome();  
})
```

```
val sortedPersons = persone.sortWith(  
    (a:Persona,b:Persona) => a.getNome() < b.getNome() );
```

```
val sortedPersons = persone.sortBy( (p:Persona) => p.getNome() );
```

```
val sortedPersons = persone.sortBy( _.getNome() );
```



E NEGLI ALTRI LINGUAGGI...?

- In Kotlin
 - la «fabbrica di comparatori» è embedded nel metodo **compareBy**
 - l'ordinamento «in place» via lambda lo fa il metodo **sortWith**

```
persone.sortWith(new MyComp()); // classico
```

Kotlin

```
persone.sortWith( object: Comparator<Persona> { // object anonimo
    override fun compare(a: Persona, b: Persona) : Int {
        if (a.getNome() < b.getNome()) return -1;
        if (a.getNome() > b.getNome()) return +1;
        /* else */ return 0;
    }
});
```

```
persone.sortWith( Comparator<Counter> { a,b -> // classe anonima
    if (a.getValue()%24 < b.getValue()%24) -1 else
    if (a.getValue()%24 > b.getValue()%24) +1 else
    0;
});
```



E NEGLI ALTRI LINGUAGGI...?

- In Kotlin

- la «fabbrica di comparatori» è embedded nel metodo `compareTo`
- l'ordinamento «in place» via lambda lo fa il metodo `sortWith`
- l'ordinamento *su nuovo array* lo fa l'analogo metodo `sortedWith`

```
persone.sortWith(new MyComp()); // classico
```

Kotlin

```
persone.sortWith( compareTo { p -> p.getNome() } );
```

```
persone.sortWith( compareTo { it.getNome() } );
```

```
val mySorted = persone.sortedWith( compareTo {  
                                     it.getCognome().length } );
```



E NEGLI ALTRI LINGUAGGI...?

- In Kotlin

- nel caso in cui basti specificare la proprietà su cui ordinare, senza altre elaborazioni, si usano i metodi-scorciatoia **sortedBy** e **sortedByDescending** che accettano direttamente *l'estrattore*
- NB: si ricorda che esiste anche il metodo **sorted** che opera con l'ordinamento naturale, senza argomenti

```
val mySorted = persone.sortedBy { it.getNome() };  
val mySorted = persone.sortedByDescending { it.getCognome() };  
val stndSorted = persone.sorted;
```

Kotlin



COMPARATORI MULTIPLI NEGLI ALTRI LINGUAGGI

- In C#

- la coppia di metodi **OrderBy** / **ThenBy** consente di comporre facilmente ordinamenti multipli *su collection* tramite lambda:

```
IOrderedEnumerable<Persona> listaOrdinata =  
    listaPersone.OrderBy( p => p.GetEta() );  
  
IOrderedEnumerable<Persona> listaOrdinata =  
    listaPersone.OrderBy( p => p.GetCognome() )  
                .ThenBy( p => p.GetNome() );  
  
IOrderedEnumerable<Persona> listaOrdinata =  
    listaPersone.OrderBy( p => p.GetCognome() )  
                .ThenBy( p => p.GetNome() )  
                .ThenBy( p => p.GetEta() );
```

C#

- NB: lavorare su array è possibile, ma **OrderBy** restituisce comunque un **IOrderedEnumerable**: quindi il risultato va ri-convertito esplicitamente in array tramite **ToArray()**

COMPARATORI MULTIPLI NEGLI ALTRI LINGUAGGI

- In Scala

- l'approccio è diverso: il metodo tipizzato `Ordering.on` accetta una lambda «a tuple di proprietà» di cardinalità variabile

```
Sorting.quickSort(elencoPersone) (  
    Ordering[(String,String)].on(p => (p.cognome, p.nome) ) )  
  
Sorting.quickSort(elencoPersone) (  
    Ordering[(String,Int)].on(p => (p.cognome, p.eta) ) )
```

Scala

- una tupla è una struttura del tipo `(item1,item2,...)` il cui tipo è `(tipoItem1,tipoItem2,...)`: quindi la tupla `(cognome, nome)` ha tipo `(string, string)` mentre la tupla `(cognome, età)` ha tipo `(string, int)`
- il metodo `on` ordina *sugli item della tupla* nel loro ordine: ad esempio, la tupla `(cognome, età)` ordina per cognome e in subordine per età
- NB: il tratto `Ordering` espone anche i metodi `thenComparing` di Java, ma il loro uso diretto causa spesso difficoltà di disambiguazione alla type inference



COMPARATORI MULTIPLI NEGLI ALTRI LINGUAGGI

- In Kotlin
 - basta *specificare più lambda* nella factory `compareBy`

```
val mySorted = persone.sortedWith( compareBy(  
    { it.getNome() }, { it.getEta() } ))  
  
val mySorted = persone.sortedWith( compareBy(  
    (Persona::getNome, Persona::getEta) )
```

Kotlin

- nel caso di proprietà (**val/var**), è ancora più semplice:

```
val sortedCounters = myCounterArray.sortedWith(  
    compareBy( { it.value } ))  
  
// se nome ed età fossero proprietà anziché metodi  
val mySorted = persone.sortedWith( compareBy(  
    { it.nome }, { it.eta } )
```

Kotlin

Method reference a costruttori (Constructor reference)



METHOD REFERENCE A COSTRUTTORI

- Abbiamo visto che le method reference seguono la sintassi ***Nomeclasse::nomemethodo***
- Un caso particolare è la **method reference a costruttori**
 - è particolare perché il costruttore non ha un «vero nome» suo, quindi serve una convenzione su "come chiamarlo"
- In Java, la «grande idea» è stata chiamarlo **new**
 - sintassi: ***Nomeclasse::new***
 - OCCHIO: non significa che si stia creando qualcosa, è solo un «*bel nome evocativo*» per «riferirsi» a un costruttore
- In Kotlin, il nome... è la classe, senza nulla davanti
 - sintassi: ***::NomeClasse***

Java

Kotlin



METHOD REFERENCE A COSTRUTTORI

- Con la sintassi della method reference a costruttori

- Java: ***NomeClasse::new***

Java

- Kotlin: ***::NomeClasse***

Kotlin

si può fare riferimento a tutti i costruttori di una classe

- Per distinguerli occorre *specificare il tipo* della funzione

- Java: specificando l'interfaccia funzionale attesa

Java

- Kotlin: specificando il tipo strutturale atteso

Kotlin

- Più precisamente, in base alla lista degli argomenti:

- Java: il tipo è ***Supplier<T>***, ***Function<U,T>***, etc.

Java

- Kotlin: il tipo è ***() ->T***, ***(U) ->T***, ***(U,V) ->T***, etc.

Kotlin



METHOD REFERENCE A COSTRUTTORI

- Più in dettaglio:

- in Java, per una classe T:

Java

- il tipo del costruttore a 0 argomenti è **Supplier<T>**
 - il tipo del costruttore a 1 argomento U è **Function<U, T>**
 - il tipo del costruttore a 2 argomenti U, V è **BiFunction<U, V, T>**
 - per costruttori a 3+ argomenti bisogna definirsi le proprie interfacce:

```
interface TriFunction<T, U, V, R> {  
    R apply(T t, U u, V v);  
}
```

- in Kotlin, il tipo è strutturale, quindi il tipo del costruttore è semplicemente **(Tipo1, Tipo2, ...) -> T**

Kotlin

- il costruttore a 0 argomenti usa la lista vuota: **() -> T**

ESEMPIO (1/4)

- Si supponga di avere una classe **Persona** con 5 costruttori:
 - la versione Kotlin, del tutto simile, non viene mostrata per brevità

```
class Persona{  
    private String cognome, nome;  
    private int eta;  
    private boolean sex;  
  
    public Persona(String nome, String cognome, int eta, boolean sex){  
        this.nome=nome; this.cognome=cognome; this.eta=eta;  
        this.sex=sex;  
    }  
  
    public Persona(String nome, String cognome, int eta){  
        this(nome, cognome, 1, true); }  
  
    public Persona(String cognome, String nome){ this(nome, cognome, 1); }  
    public Persona(String cognome){ this("boh", cognome, 1); }  
    public Persona(){ this("boh", "nnn"); }  
}
```

Java

ESEMPIO (2/4)

- Quei cinque costruttori hanno i seguenti tipi:

`Persona(String nome, String cognome, int eta, boolean sex)`

Java

tipo Java: ???

tipo Kotlin: `(String, String, Int, Boolean) -> Persona`

`Persona(String nome, String cognome, int eta)`

tipo Java: ???

tipo Kotlin: `(String, String, Int) -> Persona`

`Persona(String cognome, String nome)`

tipo Java: `BiFunction<String, String, Persona>`

tipo Kotlin: `(String, String) -> Persona`

`Persona(String cognome)`

tipo Java: `Function<String, Persona>`

tipo Kotlin: `(String) -> Persona`

`Persona()`

tipo Java: `Supplier<Persona>`

tipo Kotlin: `() -> Persona`

ESEMPIO (2/4)

- Quei cinque costruttori hanno i seguenti tipi:

`Persona(String nome, String cognome, int eta, boolean sex)`

tipo Java: `???`

tipo Kotlin: `(String,`

Non esiste un'interfaccia funzionale standard a 3 o 4 argomenti

Java

ona

`Persona(String nome, String cognome, int eta)`

tipo Java: `???`

tipo Kotlin: `(String,`

Non esiste un'interfaccia funzionale standard a 3 o 4 argomenti

`Persona(String cognome, String nome)`

tipo Java: `BiFunction<String, String, Persona>`

tipo Kotlin: `(String, String) -> Persona`

`Persona(String cognome)`

tipo Java: `Function<String, Persona>`

tipo Kotlin: `(String) -> Persona`

`Persona()`

tipo Java: `Supplier<Persona>`

tipo Kotlin: `() -> Persona`

ESEMPIO (3/4)

- Quei cinque costruttori hanno i seguenti tipi:

`Persona(String nome, String cognome, int eta, boolean sex)`

Java

tipo Java: `QuadriFunction<String,String,Integer,Boolean,Persona>`

tipo Kotlin: `(String,String,Int,Boolean)->Persona`

`Persona(String nome, String cognome, int eta)`

tipo Java: `TriFunction<String,String,Integer,Persona>`

tipo Kotlin: `(String,String,Int)->Persona`

`Persona(String nome, String cognome, String indirizzo)`

Ma, se servono, possiamo sempre definirle noi:

```
interface TriFunction<T, U, V, R> {  
    R apply(T t, U u, V v);  
}
```

`Persona(String nome, String cognome, String indirizzo)`

```
interface QuadriFunction<T, U, V, W, R> {  
    R apply(T t, U u, V v, W w);  
}
```

`Persona()`

ESEMPIO (4/4)

- Ora possiamo catturarli con opportune method reference:

```
Supplier<Persona>          ctor0 = Persona::new;  
Function<String,Persona>   ctor1 = Persona::new;  
BiFunction<String,String,Persona> ctor2 = Persona::new;  
  
// nostre interfacce funzionali «custom»  
TriFunction<String,String,Integer,Persona>  
                                ctor3 = Persona::new;  
QuadriFunction<String,String,Integer,Boolean,Persona>  
                                ctor4 = Persona::new;
```

Java

```
val ctor0: () -> Persona = ::Persona;  
val ctor1: (String) -> Persona = ::Persona;  
val ctor2: (String,String) -> Persona = ::Persona;  
val ctor3: (String,String,Int) -> Persona = ::Persona ;  
val ctor4: (String,String,Int,Boolean) -> Persona = ::Persona;
```

Kotlin

A COSA SERVONO?

- Method reference a costruttori sono utili quando occorre passare a un'altra funzione il «*modo di costruire qualcosa*»
 - una factory «parametrica» che costruisce «cose» secondo un suo schema, dove però «cosa» costruire è un parametro
 - *scenario tipico in Java: negli stream di operazioni*
 - ESEMPIO: il metodo factory **Stream.toArray** costruisce un array, ma ha bisogno di sapere esattamente *di che tipo*
 - riceve la reference al costruttore del tipo di array richiesto
 - per un array di interi, **Integer[]::new**
 - per un array di stringhe, **String[]::new**
 - per un array di Persona, **Persona[]::new**

Java

Iterazione interna



ITERAZIONE INTERNA

- Le *lambda expression* non servono solo nei comparatori: un altro caso di grande rilevanza è l'*iterazione interna*
- È un cambio di paradigma: *cambia il modo stesso di concepire un ciclo su una collection*
 - anziché una struttura di controllo (**for**, **while**) che *comanda come iterare* su una struttura dati, *si chiede alla struttura di ripetere su se stessa una operazione ricevuta come lambda*
- A tal fine, le collection (e talvolta anche gli array) definiscono un *metodo foreach (operation)*
 - Java: **forEach** (non su array)
 - C#: **ForEach** (anche su array, come static method di `Array`)
 - Scala: **foreach** (anche su array)
 - Kotlin: **forEach** (anche su array)

Java

C#

Scala

Kotlin



ITERAZIONE INTERNA

- L'iterazione interna
 - **astrae dal controllo**: non dobbiamo più preoccuparci di governare noi l'iterazione in prima persona, "saprà la collection come fare"
 - astraendo dal controllo, rende possibile **cambiare l'algoritmo di iterazione interno in modo trasparente all'esterno**
 - potenzialmente, potrebbe anche agire in parallelo....
- Filosoficamente, abitua a uno **stile più funzionale**
 - **uscire dall' "ossessione del controllo"**: non focalizzarsi in primis sul *come* fare le cose, ma su cosa si desidera ottenere
 - **stile di specifica dichiarativo** anziché imperativo
 - maggiore espressività & leggibilità, maggiori gradi di libertà futuri



ESEMPIO 1

OBIETTIVO: stampare tutte le persone di una collezione

Java

Approccio classico: iterazione *esterna*

```
for(Persona p : persone)
    System.out.println(p);
```

John	Doe	ha	25	anni
Jane	Doe	ha	45	anni
Anne	Bee	ha	31	anni
Jane	Doe	ha	22	anni

Nuovo approccio: iterazione *interna*

```
persone.forEach( p -> System.out.println(p) );
```

o, meglio, con *method reference*:

```
persone.forEach( System.out::println );
```

La *lambda operation* è un *Consumer* che consuma gli elementi uno ad uno.

- NB: in Java, *persone* dev'essere una *Collection*, *non un array*

Per iterare in modo interno su array occorre quindi prima convertirli in lista:

```
Arrays.asList(arrayPersone).forEach(System.out::println);
```

ESEMPIO 2

Java

OBIETTIVO: calcolare l'età media di un elenco (array) di persone

In teoria, verrebbe da scrivere così

```
int ageSum = 0;
Arrays.asList(persone).forEach(p -> ageSum += p.getEta());
System.out.println("età media = " + ageSum/persone.length);
```

MA *in Java non è possibile modificare variabili all'interno di una lambda*
(le variabili di «chiusura» devono essere `final` o *effectively final*)

La soluzione è incapsulare la variabile in un qualche wrapper

- non può essere `Integer`, perché non permette modifiche al valore incapsulato
- conviene definire al volo una *classe anonima* estendendo `Object`:

```
var wrapper = new Object(){ int ageSum = 0; };
Arrays.asList(persone).forEach(p -> wrapper.ageSum += p.getEta());
System.out.println("media = " + wrapper.ageSum/persone.length);
```

Così, la variabile (riferimento) in sé non viene modificata, ma tramite essa si registra comunque la variazione richiesta.

GLI ESEMPI NEGLI ALTRI LINGUAGGI

In C#

C#

- per gli array esiste il **metodo statico tipizzato `ForEach`** nella **libreria `Array`**: riceve due argomenti: l'array su cui operare e la lambda corrispondente (una **`Action`**)

```
Array.ForEach<Persona>(persone, p => Console.WriteLine(p));
```

- a differenza di Java, è possibile anche modificare le variabili nella lambda

```
int ageSum = 0;
```

```
Array.ForEach<Persona>(persone, p => ageSum += p.GetEta());
```

```
Console.WriteLine("media = " + ageSum/persone.Length);
```

- è inoltre disponibile il classico **metodo `ForEach`** (a un argomento) per le collection

```
List<Persona> lista = persone.ToList();
```

```
lista.ForEach(p => Console.WriteLine(p));
```

- In più, C# definisce alcuni metodi *fluent* per elaborare la lista, come il filtro **`FindAll`**: questo esempio stampa solo le persone di età minore di 40 anni:

```
lista.FindAll(p => p.GetEta() < 40)
```

```
.ForEach(p => Console.WriteLine(p));
```

In Java lo faremo con gli
stream di operazioni



GLI ESEMPI NEGLI ALTRI LINGUAGGI

In Scala

Scala

- gli array sono assimilati a collection, quindi è applicabile direttamente anche su di essi il classico metodo **ForEach**:

```
persone.foreach(println(_)); // con parametro implicito
```

- come in C# e a differenza di Java, è possibile modificare le variabili nella lambda

```
var ageSum = 0;  
persone.foreach( p => ageSum += p.eta );  
println("media = " + ageSum/persone.size);
```

- ovviamente, il metodo **foreach** opera anche sulle collection

```
val lista = persone.toList  
lista.foreach(println);
```

- anche Scala definisce alcuni metodi *fluent* per elaborare la lista, come il filtro **filter**, qui utilizzato con la comoda notazione a parametri impliciti (placeholder):

```
lista.filter(_.eta<40)  
    .foreach(println(_));
```

In Java lo faremo con gli
stream di operazioni



GLI ESEMPI NEGLI ALTRI LINGUAGGI

In Kotlin

Kotlin

- come in Scala, anche qui gli **array** sono assimilati a **collection**:

```
persone.forEach( {println(it)} ); // con parametro implicito
```

- anche qui è possibile modificare le variabili nella lambda:

```
var ageSum = 0;  
persone.forEach( { p -> ageSum += p.eta } );  
println("media = " + ageSum/persone.size);
```

- ovviamente, il metodo **foreach** opera anche sulle **collection**

```
val lista = persone.toList()  
lista.foreach( {println(it)} );
```

- anche Kotlin definisce alcuni metodi *fluent* per elaborare la lista, come il filtro **filter**, qui utilizzato con la comoda notazione a parametri impliciti (placeholder):

```
lista.filter( {it.eta<40} )  
.foreach( {println(it)} );
```

In Java lo faremo con gli
stream di operazioni



COMPOSIZIONE DI FUNZIONI

- Alcuni linguaggi consentono di *comporre due lambda* per definire una nuova funzione composta
 - in Java: **andThen** e **compose** (solo su tipi funzione omogenei)
 - in Scala: **andThen** e **compose** (solo su funzioni a 1+ argomenti)
- Più precisamente:
 - **f.andThen(g)** definisce la funzione «sequenza» $g(f(...))$
 - **f.compose(g)** definisce la funzione «composta» $f(g(...))$

```
IntUnaryOperator mulBy3 = x -> x * 3;  
IntUnaryOperator incBy1 = x -> x + 1;  
  
var y = mulBy3.andThen(incBy1).applyAsInt(4); // dà (3x4)+1 = 13  
var z = mulBy3.compose(incBy1).applyAsInt(4); // dà 3x(4+1) = 15
```

Java

```
val mulBy3 : (Int) => Int = (x) => x*3;  
val incBy1 : (Int) => Int = (x) => x+1;  
  
val y = mulBy3.andThen(incBy1)(4) // dà (3x4)+1 = 13  
val z = mulBy3.compose(incBy1)(4) // dà 3x(4+1) = 15
```

Scala



COMPOSIZIONE DI FUNZIONI ANALOGIE & DIFFERENZE

- In Java

- **andThen** e **compose** possono essere usate per **comporre solo tipi funzione omogenei** (ad esempio, due **IntUnaryOperator**, due **IntConsumer**, etc.): non è ammesso comporre entità diverse, anche quando la composizione avrebbe senso
- però, **andThen** è definito anche sui **consumer**, intendendo che entrambi «mangiano» lo stesso valore, in sequenza

- In Scala

- **andThen** e **compose** possono essere usate per comporre **anche tipi funzione disomogenei**, purché la composizione abbia senso
- però, la composizione non è ammessa con i consumer (funzioni a codominio **Unit**), in quanto appunto «consumano» il valore, neppure nella forma **andThen** supportata invece da Java

ESEMPIO

- Fase 1: definizione e test di due consumer

Java

```
IntConsumer printIfEven = (x) -> {  
    if(x%2==0) System.out.println("Even " + x);};  
  
IntConsumer printIfOdd = (x) -> {  
    if(x%2!=0) System.out.println("Odd " + x);};  
  
List.of(22,21,13,42,-5,-6).forEach(  
    (Integer x) -> printIfEven.accept(x.intValue()) );  
  
List.of(22,21,13,42,-5,-6).forEach(  
    (Integer x) -> printIfOdd.accept(x.intValue()) );
```

Scala

```
val printIfEven : (Int) => Unit =  
    (x) => if(x%2==0) println("Even " + x);  
  
val printIfOdd : (Int) => Unit =  
    (x) => if(x%2!=0) println("Odd " + x);  
  
List(22,21,13,42,-5,-6).foreach(printIfEven(_))  
List(22,21,13,42,-5,-6).foreach(printIfOdd(_))
```

ESEMPIO

- Fase 1: definizione e test di due consumer

```
IntConsumer printIfEven = (x) -> {  
    if(x%2==0) System.out.println("Even " + x);};  
  
IntConsumer printIfOdd = (x) -> {  
    if(x%2!=0) System.out.println("Odd " + x);};  
  
List.of(22,21,13,42,-5,-6).forEach(printIfEven::accept);  
List.of(22,21,13,42,-5,-6).forEach(printIfOdd::accept);
```

Java

Method reference a oggetti funzione: BELLO! 😊😊
MA serve sapere il nome del metodo interno...😞😞

```
val printIfEven : (Int) => Unit =  
    (x) => if(x%2==0) println("Even " + x);  
  
val printIfOdd : (Int) => Unit =  
    (x) => if(x%2!=0) println("Odd " + x);  
  
List(22,21,13,42,-5,-6).foreach(printIfEven(_))  
List(22,21,13,42,-5,-6).foreach(printIfOdd(_))
```

Scala

ESEMPIO

- Fase 2: composizione fra consumer
 - in Java, possibile **andThen** fra consumer (non ammessa in Scala)
 - ovviamente, invertendo l'ordine qui non cambia niente..

```
printIfOdd.andThen(printIfEven).accept(4); // Even 4  
printIfOdd.andThen(printIfEven).accept(5); // Odd 5  
printIfEven.andThen(printIfOdd).accept(7); // Odd 7  
printIfEven.andThen(printIfOdd).accept(6); // Even 6
```

Java

- Fase 3: composizione fra entità diverse, purché sensata
 - in Scala, possibile mixare consumer e funzioni sia con **andThen**...
 - ovviamente, qui l'ordine è fondamentale: non si può fare viceversa

```
mulBy3.andThen(printIfEven)(4) // 12  
mulBy3.andThen(printIfEven)(5) // non stampa nulla  
mulBy3.andThen(printIfOdd)(7) // 21  
mulBy3.andThen(printIfOdd)(6) // non stampa nulla
```

Scala

ESEMPIO

- Fase 4: composizione fra entità diverse, purché sensata
 - in Scala, possibile mixare consumer e funzioni **anche con compose**
 - ovviamente, qui l'ordine è invertito rispetto al caso **andThen!**

```
printIfEven.compose(incBy1)(4) // non stampa nulla  
printIfEven.compose(incBy1)(5) // 6  
printIfOdd.compose(incBy1)(7) // non stampa nulla  
printIfOdd.compose(incBy1)(6) // 7
```

Scala

- Fase 3: composizione fra entità diverse, purché sensata
 - in Scala, possibile mixare consumer e funzioni **sia con andThen...**
 - ovviamente, qui l'ordine è fondamentale: non si può fare viceversa

```
mulBy3.andThen(printIfEven)(4) // 12  
mulBy3.andThen(printIfEven)(5) // non stampa nulla  
mulBy3.andThen(printIfOdd)(7) // 21  
mulBy3.andThen(printIfOdd)(6) // non stampa nulla
```

Scala



ITERAZIONI COMPOSITE

- Le funzioni composte sono particolarmente comode in accoppiata con l'iterazione interna, *foreach*
 - quando una singola lambda come *operation* non basta più, se ne può creare una composta tramite **andThen** e/o **compose**
 - con l'unico limite di *non portare agli estremi la type inference*
- Esempio: ancora Persone
 - immaginiamo di ampliare la classe **Persona** aggiungendo **due semplici consumer** che filtrino e stampino solo:
 - una persona «giovane» (under 25) → **printIfYoung**
 - una persona «matura» (over 40) → **printIfMature**
 - OBIETTIVO: stampare le persone tranne quelle fra 25 e 40 anni
 - IDEA: *comporre opportunamente*, tramite **andThen**, i due consumer



ITERAZIONI COMPOSITE

- Esempio (segue)
 - OBIETTIVO: stampare le persone tranne quelle fra 25 e 40 anni
 - IDEA: comporre `printIfYoung` e `printIfMature`
- Prima possibilità
 - definiamo due alias di tipo `Consumer<Persona>`

```
Consumer<Persona> printIfOver40 = Persona::printIfMature;  
Consumer<Persona> printIfUnder25 = Persona::printIfYoung;  
elencoPersone.forEach(  
    printIfOver40.andThen(printIfUnder25));
```

stampa completa

```
John Doe ha 25 anni  
Jane Doe ha 45 anni  
Anne Bee ha 31 anni  
Jane Doe ha 22 anni
```

stampa composta

```
Jane Doe ha 45 anni  
Jane Doe ha 22 anni  
(non stampa la 31enne)  
(non stampa il 25enne)
```

Java



ITERAZIONI COMPOSITE

- Perché questi alias?
 - perché l'uso diretto di `printIfYoung` e `printIfMature` metterebbe in crisi la type inference, causando **errore di compilazione**

```
elencoPersone.forEach( // NON COMPILA
```

Java

```
    Persona::printIfMature.andThen(Persona::printIfYoung) );
```

- MOTIVO: una method reference da sola non può fare da target per una chiamata di metodo, perché non dà sufficienti informazioni: è un problema di *target type non sufficientemente specificato*

TARGET TYPE

- Il compilatore deve poter dedurre *con certezza* il tipo della lambda (*target type*) e per farlo ha bisogno di informazioni adeguate *su ogni singolo passaggio*
 - non sempre le informazioni che ha bastano allo scopo
 - in particolare, servono *tipo degli argomenti* e *contesto d'uso*
- Le catene di comparatori (con **thenComparing**) e il concatenamento di funzioni (con **andThen**) *spingono al limite* le capacità inferenziali del compilatore
 - MOTIVO: *spesso si conosce il tipo atteso del risultato nel suo complesso, ma non dei singoli pezzi (se sono più d'uno)*
 - ergo, spesso una singola inferenza riesce, una catena no
 - le method reference aggravano il problema, in quanto forniscono solo il *nome* del metodo ma *non gli argomenti*

TARGET TYPE

- Il problema
 - se le informazioni in mano al compilatore non bastano per determinare la *signature di ogni singolo pezzo*, la type inference fallisce
 - per risolvere, bisogna dare al compilatore tali informazioni *per qualche altra via*
- Possibili cure
 - inserire un cast al *tipo di method reference specifica*, così da dargli «un aiutino» al punto giusto (Java only)
 - spostare la **andThen** in una *funzione ausiliaria*, i cui argomenti *tipizzati* diano al compilatore l'informazione che manca
 - incapsulare la *prima method reference in una funzione-identità* (es. `itself`) il cui argomento, di nuovo, abbia il solo scopo di *completare* l'informazione di tipo.



ITERAZIONI COMPOSTE REFACTORING per target type

- Versione che non compila:

```
elencoPersone.forEach(  
    Persona::printIfMature. andThen (Persona::printIfYoung)) ;
```

Java

- Chiamata riformulata tramite cast

```
elencoPersone.forEach(  
    ((Consumer<Persona>) Persona::printIfMature)  
        . andThen (Persona::printIfYoung) ) ;
```

Java

- **andThen** spostata in una funzione ausiliaria (es. **combina**)

```
elencoPersone.forEach(  
    combina(Persona::printIfMature, Persona::printIfYoung)) ;
```

Java

- Prima method reference incapsulata in una funzione-identità

```
elencoPersone.forEach(  
    itself(Persona::printIfMature)  
        . andThen (Persona::printIfYoung) ) ;
```

Java



ITERAZIONI COMPOSITE REFACTORING per target type

- Versione che non compila:

```
elencoPersone.forEach(  
    Persona::printIfMature.andThen(Persona::printIfYoung));
```

Java

- **Combina** if applica due funzioni a target type

```
private static <T> Consumer<T>  
    combina(Consumer<T> f1, Consumer<T> f2) {  
        return f1.andThen(f2);  
    }
```

```
        printIfMature()  
        .andThen(printIfYoung) );
```

Java

- **andThen** applica la seconda funzione alla prima in una funzione ausiliaria (es. **combina**)

```
elencoPersone.forEach(  
    combina(Persona::printIfMature,
```

```
private static <T> Consumer<T>  
    itself(Consumer<T> f) {  
        return f;  
    }
```

- Prima method reference incapsulata

```
elencoPersone.forEach(  
    itself(Persona::printIfMature)  
        .andThen(Persona::printIfYoung));
```

Java



ITERAZIONI COMPOSITE in SCALA

- In Scala

- non esistendo method reference, si utilizzano in loro vece i parametri impliciti, scrivendo:

`_.printIfYoung()` al posto di `Persona::printIfYoung`

`_.printIfMature()` al posto di `Persona::printIfMature`

- non potendo combinare consumer, occorre cambiare il tipo delle due funzioni `printIfYoung` e `printIfMature`

→ adottiamo il pattern *cascading*, restituendo *la persona stessa*

```
case class Persona(val nome:String, val cognome:String, val eta:Int){  
  def printIfMature() : Persona = { if(eta>40) println(this); this; }  
  def printIfYoung()  : Persona = { if(eta<25) println(this); this; }  
}
```



ITERAZIONI COMPOSITE in SCALA

- In Scala

- l'esempio assume allora la seguente forma:

```
val printIfOver40 = (p:Persona) => p.printIfMature();  
// OR: val printIfOver40 : (Persona) => Persona = _.printIfMature();  
val printIfUnder25 = (p:Persona) => p.printIfYoung();  
// OR: val printIfUnder25 : (Persona) => Persona = _.printIfYoung();  
persone.foreach(printIfOver40.andThen(printIfUnder25)); // OK
```

Scala

- come prevedibile, la seguente forma NON compila:

```
persone.foreach((_.printIfMature()).andThen(_.printIfYoung()));  
// NON COMPILA: missing parameter for expanded function
```

- CURA: il cast in Scala ha la forma **asInstanceOf[T]**, che però non è applicabile al placeholder **_**
Quindi, restano solo le altre due opzioni.



ITERAZIONI COMPOSITE in SCALA

REFACTORING per target type

- Versione che non compila:

```
persone.foreach(  
    (_.printIfMature()).andThen(_.printIfYoung()));
```

Scala

- **andThen** spostata in una funzione ausiliaria (es. **combina**)

```
persone.foreach(  
    combina(_.printIfMature(), _.printIfYoung()));  
  
def combina(a: (Persona)=>Persona, b: (Persona)=>Persona):  
    (Persona)=>Persona = a.andThen(b);
```

Scala

- Prima method reference incapsulata in una funzione-identità

```
persone.foreach(  
    itself(_.printIfMature()).andThen(_.printIfYoung()));  
  
def itself(p: (Persona)=>Persona) :  
    (Persona)=>Persona = p;
```

Scala